

SQL Assignment — Group 6: Constraints

Foreign Keys and Views

Group 6 — Constraints, Foreign Keys, and Views

1. Restaurant table with NOT NULL + PRIMARY KEY

Create a table called *Restaurant* with columns: *id*, *name*, *location*, and *cuisine*. Apply *NOT NULL* to *name* and *location*, and make *id* the *PRIMARY KEY*.

```
CREATE TABLE Restaurant (  
  id          INT PRIMARY KEY,  
  name       VARCHAR(100) NOT NULL,  
  location   VARCHAR(100) NOT NULL,  
  cuisine    VARCHAR(50)  
);
```

Explanation: *PRIMARY KEY* on *id* makes it unique and automatically non-null. The explicit *NOT NULL* constraints on *name* and *location* mean any *INSERT* or *UPDATE* that leaves these fields empty will be rejected by the database, enforcing that every restaurant has at least a name and a location on record. *cuisine* is left nullable/optional.

2. FoodOrder table with FOREIGN KEY to Restaurant

Create a table called *FoodOrder* with columns: *order_id*, *restaurant_id*, *user_id*, and *order_total*. Set *order_id* as *PRIMARY KEY*, and add a *FOREIGN KEY* on *restaurant_id* referencing *Restaurant(id)*.

```
CREATE TABLE FoodOrder (  
  order_id      INT PRIMARY KEY,  
  restaurant_id INT,  
  user_id       INT,  
  order_total   DECIMAL(10,2),  
  FOREIGN KEY (restaurant_id) REFERENCES Restaurant(id)  
);
```

Explanation: *order_id* uniquely identifies each order as the primary key. The *FOREIGN KEY (restaurant_id) REFERENCES Restaurant(id)* constraint ensures every order must point to a restaurant that actually exists in the *Restaurant* table, preventing orphaned orders tied to a non-existent restaurant.

3. SpotifyUser — 6 rows, UNIQUE username, NOT NULL email

Insert 6 rows into a table called *SpotifyUser* with columns: *user_id*, *username*, *email*, and *subscription_type*. Ensure *username* is *UNIQUE* and *email* is *NOT NULL*.

```
CREATE TABLE SpotifyUser (  
  user_id        INT AUTO_INCREMENT PRIMARY KEY,  
  username       VARCHAR(50) UNIQUE,  
  email          VARCHAR(100) NOT NULL,  
  subscription_type VARCHAR(20)  
);  
  
INSERT INTO SpotifyUser (username, email, subscription_type) VALUES
```

```
('musiclover_riya', 'riya@example.com', 'Premium'),
('beatking_karan', 'karan@example.com', 'Free'),
('aisha_tunes', 'aisha@example.com', 'Premium'),
('dj_rohan', 'rohan@example.com', 'Family'),
('podcast_neha', 'neha@example.com', 'Free'),
('lofi_sam', 'sam@example.com', 'Premium');
```

Explanation: UNIQUE on username means the database will reject any attempt to insert a second user with a username already in use, preventing duplicate handles. NOT NULL on email guarantees every user record has a contactable email address on file — inserting a row without one would fail.

4. CREATE VIEW TopSpendersView

Create a VIEW named *TopSpendersView* that shows usernames and *order_total* from a *FoodOrder* table where *order_total* is greater than 1000.

```
-- Assumes FoodOrder also has (or is joined to get) a username; here we join
-- FoodOrder.user_id to SpotifyUser.user_id purely as an example of pulling
-- a readable username alongside the order_total.
CREATE VIEW TopSpendersView AS
SELECT su.username, fo.order_total
FROM FoodOrder fo
JOIN SpotifyUser su ON fo.user_id = su.user_id
WHERE fo.order_total > 1000;
```

Explanation: A VIEW is a saved, reusable query that behaves like a virtual table — *TopSpendersView* doesn't store its own data, but every time it's queried it re-runs the underlying SELECT, returning only orders where *order_total* > 1000 along with the matching username. This is useful for hiding query complexity behind a simple, reusable name.

5. Query TopSpendersView for users who spent > 2000

List all users from *TopSpendersView* who have spent more than 2000, sorted by *order_total* descending. Use only the view, not the original *FoodOrder* table.

```
SELECT *
FROM TopSpendersView
WHERE order_total > 2000
ORDER BY order_total DESC;
```

Explanation: Because *TopSpendersView* is queried exactly like a normal table, this statement applies an additional filter (*order_total* > 2000, narrower than the view's own > 1000 filter) directly on top of it, then sorts by *order_total* from highest to lowest — all without referencing the original *FoodOrder* table directly.